Human Activity Recognition from Wi-Fi CSI Doppler Spectrograms: A CNN Baseline and a Contrastive Extension for Cross-Domain Generalization

Marco Carraro[†] Francesco Vezzani[†]

***Abstract*—We study device-free human activity recognition (HAR) from the Doppler spectrograms of Wi-Fi channel-state information (CSI), using the released traces of the SHARP dataset [1], [2]. Starting from the Inception-style convolutional baseline, trained only on measurement set $S1$ and evaluated on the six unseen sets $S2$–$S7$. We add and measure, one change at a time, four strategies for closing the cross-domain gap without the hand-engineered front-end of the reference system: (i) supervised contrastive pre-training, with a linear probe on the frozen representation; (ii) a slower optimizer schedule; (iii) an overlapping sliding window at the stride used by the dataset authors and (iv) multi-antenna soft decision fusion, averaging the per-antenna probability vectors of each physical time window with no retraining and no extra parameters. The last two compose into our best configuration, taking the same network from 48.2% to $\mathbf{72.7}\%$ and beating the single-component ablation of the reference system on every unseen set. Contrastive pre-training helps in the data-poor regime but stops helping once the sliding window supplies substantially more training windows. The residual gap to the full reference pipeline is attributable to its strongest-path phase sanitization, which we do not implement.**

I. Introduction

Contactless sensing of human activity is valuable in healthcare, security and ambient monitoring. Commodity Wi-Fi devices can be repurposed as radar-like sensors: as a person moves, their body induces micro-Doppler shifts captured in the channel-state information (CSI) estimated by the receiver, without recording any video. After processing, the CSI becomes a Doppler spectrogram, a time $\times$ velocity image, whose appearance changes markedly with the activity, so recognition can be cast as image classification with a convolutional neural network (CNN) [1].

The central difficulty is generalization. Different people move differently, and different rooms, even the same room on different days, distort the signal differently. So a model that classifies perfectly in its training environment may collapse on a new one. The project proposal raises exactly this challenge and suggests a contrastive / self-supervised approach to obtain subject- and environment-independent features. Prior systems close the gap mostly through signal processing, robust phase sanitization and multi-antenna fusion, at the cost of a heavy, hand-engineered pipeline [1].

We tackle five-class Wi-Fi HAR under a strict cross-domain protocol (train on one environment, test on six unseen ones) and ask how much of the gap can be closed without that front-end: by learning invariance with Supervised Contrastive Learning (SupCon) [3], by re-using the receiver diversity the dataset already provides, and by feeding the network more of the signal it already has. Every change is implemented and measured one at a time, on the same baseline and protocol, so each accuracy figure is attributable to one design decision. The contributions are:

- **Baseline (M. Carraro).** A plain reimplementation of the dataset pipeline and baseline Inception-style CNN, with a cross-domain protocol (train on $S1$, test on $S2$–$S7$) and a per-set error breakdown.
- **Contrastive representation (F. Vezzani).** A batch-normalized encoder pre-trained with a supervised-contrastive loss and domain-randomization augmentations, evaluated with a linear probe on the frozen features: it improves every set when training data is scarce, and stops helping once it is not.
- **Multi-antenna decision fusion (M. Carraro).** A group identifier recovers which windows are the same physical time window seen by the four antennas, so their predictions can be fused by averaging class probabilities.
- **Front-end and optimization engineering.** A sliding window at the stride used by the SHARP authors (30 samples, 91% overlap) replaces the naive segmentation, affordable only through a two-pass, memory-bounded dataset builder. A controlled comparison of optimizer schedules shows that, with a single training environment, fitting the training set better does not imply generalizing better.

Combining the last two takes the same architecture from 48.2% to $\mathbf{72.7}\%$. We use the full SHARP system [1] as the reference upper bound and discuss, quantitatively, which part of the gap each strategy removes and which part is left.

II. Related Work

Wi-Fi sensing and the SHARP system. Device-free HAR from commodity Wi-Fi has matured rapidly thanks to CSI

[†]Department of Information Engineering, University of Padova, Italy,
email: marco.carraro.23@studenti.unipd.it, [Mat. 2159568]
email: francesco.vezzani.1@studenti.unipd.it, [Mat. 2159567]

extraction tools and public datasets. SHARP [1] recognizes activities from the Doppler information in IEEE 802.11ac CSI and targets environment and person independence through two ingredients: (i) a robust phase-sanitization step that isolates the multipath components and adopts the strongest path as phase reference, and (ii) a decision-fusion scheme that runs a compact Inception-style CNN on each of the four receiver antennas and combines their decisions. On the seven measurement sets of the accompanying dataset [2] the full system exceeds 95% even on rooms, days and persons never seen at training time. Those results rest on a heavy, hand-engineered front-end: the paper's own ablation shows that replacing the strongest-path sanitization with the standard reference-antenna method makes the same CNN collapse to 39–60% on the unseen sets, i.e. much of the robustness is carried by the preprocessing rather than the learned representation. Our work keeps the minimal front-end and asks how much cross-domain robustness can be recovered elsewhere: on the representation side (contrastive pre-training) and on the decision side, by re-implementing SHARP's antenna fusion, which, unlike sanitization, costs nothing at training time and turns out to be worth more than any other change we made.

Contrastive representation learning. Contrastive methods learn representations by pulling together "views" that should be similar and pushing apart those that should differ. SimCLR [4] established the self-supervised recipe of contrasting two augmented views against in-batch negatives with a temperature-scaled loss on L_2-normalized embeddings. Supervised Contrastive Learning (SupCon) [3] extends it to the labelled setting (all samples sharing the anchor's label become positives) producing tighter, more class-consistent clusters than cross-entropy. We adopt SupCon because its notion of "positive" maps onto our goal: two windows of the same activity should be neighbours whatever environment, person or antenna they come from. Combining it with explicit domain-randomization augmentations for cross-domain Wi-Fi HAR on the SHARP data is the extension we contribute, and Section V quantifies both its gains and the regime in which they vanish.

III. Signals and Features

A. The SHARP data

We use the pre-extracted Doppler traces released with the SHARP dataset [2] (IEEE 802.11ac, 80 MHz): arrays of shape $(T, 100)$ (T time steps, 100 Doppler-velocity bins) one per receiving antenna (`stream_1`…`stream_4`) of a four-antenna monitor. They map to seven measurement sets $S1$–$S7$, each a distinct (environment, monitor position, person, day) combination. Only $S1$ is used for training, instead the others explore generalization to an unseen day ($S2$), an unseen person ($S3$), blocked line-of-sight ($S4$, $S5$), a different room ($S6$), and new room, day and person at once ($S7$).

B. From traces to windows

We use the five core activities present in every environment: **E** (empty room), **J** (jumping), **L** (sitting still), **R** (running), **W** (walking), matching the five-class task of [1]. Each trace is cut into windows of $W_{\text{len}} = 340$ time samples (≈ 2 s) advancing by a stride S, plus a channel dimension with inputs of shape $1 \times 340 \times 100$. Each antenna stream is an independent labelled sample, as SHARP does at training time. The antenna identity is re-used only at decision time (Section IV-C).

Windowing strategy. The reference configuration uses $S = W_{\text{len}} = 340$, i.e. non-overlapping windows: simplest, but wasteful. A trace of length T yields only $\lfloor T/340 \rfloor$ examples and any pattern straddling a boundary is cut in half. The alternative is a sliding window: after prototyping $S = 170$ we adopted $S = 30$, the value hard-coded in the SHARP authors' own dataset script, 91% overlap and 152,320 windows from the same recordings. They are strongly correlated, so the number of independent examples grows far less than tenfold, and a window and its neighbour can fall on opposite sides of the random train/val split, making the validation loss optimistic and perturbing when early stopping fires. The test figures are unaffected.

Group identifiers. The four streams of a recording have identical length, so window k of each starts at sample kS of the same event. The parser therefore keeps the full recording token (`"J2"`, repetition number included) and the loader emits a group id `env_token_k` per window: windows sharing it are the same physical time window seen by different antennas, which is what makes Section IV-C possible. The id travels through the split with the labels, and the test `DataLoader` uses `shuffle=False` to stay aligned with it.

Memory-bounded loader. At $S = 30$ the windows no longer fit comfortably in a Colab runtime. The rewritten loader makes two passes: the first counts the windows each trace will produce, the second fills one pre-allocated `float32` array and frees each trace as soon as it is windowed, loading train/val and test as two independent calls and deleting the `train_test_split` copy explicitly. Peak memory is roughly halved and the windows are identical.

C. Cross-domain split

Following the SHARP protocol the split is by environment, never random: $S1$ ($S1a, S1b, S1c$) is used for training/validation and every other set is held out. A random split would leak windows from the same person/room/day into both sides and overestimate performance. Within $S1$ we keep 85% for training and 15% for validation, 2767 / 489 / 10,308 windows for train / val / test at $S = 340$ and 31,103 / 5489 / 115,728 at $S = 30$. Grouping the four antennas collapses the test windows to 2577 and 28,932 distinct physical time windows respectively.

IV. Learning Framework

A. Baseline CNN

The baseline reproduces the reduced-Inception module of SHARP ([1], Fig. 4), shown in Fig. 1: three parallel branches (max-pool, strided conv, three stacked convs) turn the $1 \times 340 \times 100$ input into feature maps of identical size 170×50, concatenated on the channel axis ($1{+}5{+}9 = 15$), reduced to 3

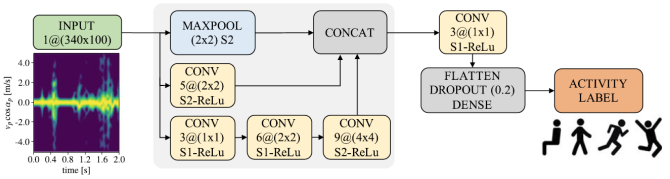


Fig. 1: Baseline single-antenna architecture of SHARP reduced-Inception module ([1], Fig. 4). `S2` denotes stride 2; every CONV is followed by a ReLU.

channels by a 1×1 convolution, flattened and classified by a dense layer after $\mathrm{Dropout}(0.2)$ leads to 128,535 parameters, exactly the count reported for the SHARP baseline CNN. Training uses cross-entropy, Adam, mini-batches of 64 and early stopping, restoring the lowest-validation-loss weights.

We compare a fast schedule (learning rate 10^{-3}, patience 7, maximum 40 epochs) with a slow schedule (10^{-4}, patience 5, maximum 100 epochs), keeping the model, seed, batch size and split fixed. Sliding-window training uses learning rate 10^{-3} and patience 12. In every case, early stopping restores the weights with the lowest validation loss.

B. Contrastive extension: Supervised Contrastive Learning

1) Motivation and augmentations: We use Supervised Contrastive Learning (SupCon) [3] to learn similar representations for the same activity and different representations for different activities, regardless of person, environment or antenna. Two augmented views are created from each window using amplitude scaling ($\mathcal{U}(0.8, 1.2)$), Gaussian noise ($\sigma = 0.05$), circular time shifts (± 40 samples), and time/Doppler masking (40 time samples and 15 velocity bins). After pre-training, the encoder is frozen and a linear classifier is trained on its output.

2) Encoder and projection head: The encoder extends the baseline's three-branch Inception block with batch normalization, a 1×1 reduction to 32 channels and a second downsampling stage ($32 \to 64 \to 128$). Global average pooling produces a 128-dimensional representation h. A projection head ($128 \to 128 \to 128$) maps h to a normalized vector $\boldsymbol{z}$ for the SupCon loss. The head is removed after pre-training, so the classifier uses h. The model has 130,952 parameters.

3) SupCon loss: For each sample i, the loss pulls it closer to samples with the same label, denoted by $P(i)$, and pushes it away from samples with different labels:

$$\mathcal{L} = \frac{1}{|I|} \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\boldsymbol{z}_i \cdot \boldsymbol{z}_p / \tau)}{\sum_{a \neq i} \exp(\boldsymbol{z}_i \cdot \boldsymbol{z}_a / \tau)}, \qquad (1)$$

where I contains samples with a same-label partner and $\tau = 0.07$. Unlike SimCLR [4], SupCon uses labels, so same-activity windows from different environments are also treated as positive pairs.

4) Training and evaluation protocol: We use the same $S1$-based split as the baseline. The encoder and projection head are pre-trained with Adam for up to 30 epochs, using weight decay 10^{-4} and early stopping. The linear probe freezes the encoder and trains a $\mathrm{Linear}(128, 5)$ classifier. Fine-tuning instead updates the full model at learning rate 10^{-4}. A t-SNE plot checks whether the learned groups depend on activity rather than environment. Pre-training and probe learning rates are 10^{-3} for non-overlapping windows and 10^{-4} for sliding windows.

C. Multi-antenna decision fusion

The dataset ships the four antenna streams separately, and both chains so far discard that structure. SHARP keeps the per-antenna training convention but fuses the four decisions at inference [1]; that step is reproduced here as pure post-processing. For a group id g, let A_g be the (up to four) antenna windows carrying it and $\boldsymbol{p}_i = \mathrm{softmax}(f(\boldsymbol{x}_i))$ the class probabilities of window i. The fused decision is

$$\hat{y}_g = \underset{c \in \{1,\ldots,5\}}{\arg\max} \; \frac{1}{|A_g|} \sum_{i \in A_g} p_{i,c}, \qquad (2)$$

i.e. a soft vote, preferred to majority voting on the hard labels because a confident antenna can outweigh two unsure ones and because it degrades gracefully when a recording provides fewer than four streams. The reference paper does not detail its own rule, so Eq. (2) is a standard instantiation rather than a reproduction.

Fusion changes the unit of evaluation from one decision per antenna to one decision per physical time window.

V. Results

We train on $S1$ and test on the unseen $S2$–$S7$. Tab. 1 reports the accuracy of each configuration per set.

A. Baseline and optimizer schedule

On the pooled unseen test set the baseline reaches $\mathbf{48.2}\%$ (test loss 1.052), from 45.5% ($S2$) to 52.3% ($S7$). Its confusion matrix (Fig. 2 (a)) has two dominant error modes: sitting still is absorbed by empty ($L \to E$: 0.76), both near-static and concentrated at zero Doppler velocity, and running/walking are heavily confused (0.39 and 0.28).

The slow schedule improves accuracy to 51.3% ($+3.1$ points) even though training accuracy is lower (0.575 instead of 0.601). With only one training environment, fitting the training set better can also mean learning more environment-specific details.

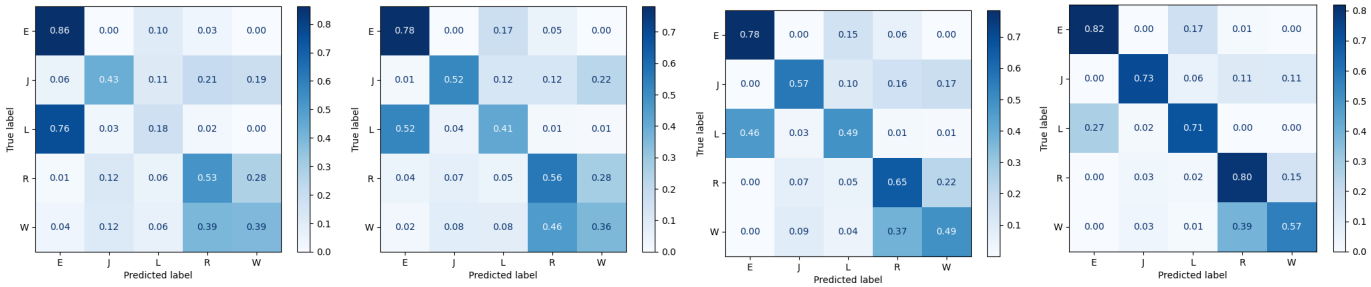
B. Contrastive representation

The linear probe on the frozen SupCon encoder raises accuracy to $\mathbf{53.1}\%$ (validation 58.3%): $+4.9$ points and an improvement on every set, the largest on $S3$ ($+8.3$, unseen person) and $S7$ ($+5.9$). Its confusion matrix (Fig. 2 (b)) sharpens running and softens $L \to E$ ($0.76 \to 0.52$). Fine-tuning then hurts, dropping to 48.4%: freezing the encoder is what generalizes instead fine-tuning re-overfits the one environment.

The t-SNE projection (Fig. 3, from the sliding-window run of Section V-C) is the most informative diagnostic. Colored by environment (right) the points are totally mixed; the SupCon objective did reach its design goal of an environment-invariant representation. Colored by activity (left) the space splits into a static region (empty, sitting) and a moving arc (jumping,

TABLE 1: Per-set accuracy (%), training on $S1$ and testing on the unseen sets, one run per column. Non-overlapping windows: baseline under the fast (10^{-3}) and slow (10^{-4}) schedules, SupCon linear probe, and fusion on the slow-schedule model. Sliding window (stride 30): baseline, SupCon probe and fusion. Fusion always re-uses the weights of the baseline to its left. SHARP columns from [1] (Tables 3 and 4).

	Non-overlapping windows				**Sliding window (stride 30)**			**SHARP [1]**	
Set	Base (fast)	SupCon	Base (slow)	+ Fusion	Base	SupCon	+ Fusion	Full	Abl.
$S2$	45.5	51.1	50.9	62.6	61.6	61.6	77.1	99.8	59.8
$S3$	51.4	59.7	55.9	68.8	63.3	63.2	77.3	99.3	58.0
$S4$	48.5	51.9	50.6	58.1	57.1	56.5	66.4	97.6	43.5
$S5$	47.5	52.9	49.8	57.5	58.0	55.4	66.4	96.6	39.2
$S6$	48.1	51.3	50.6	57.9	58.3	57.0	75.2	99.8	59.4
$S7$	52.3	58.2	51.5	68.9	64.7	66.1	70.7	96.0	60.3
Overall	48.2	53.1	51.3	61.5	60.1	59.5	**72.7**	∼98	∼53



(a) baseline, non-overlapping (b) + SupCon linear probe (c) baseline, sliding window (d) + 4-antenna fusion

Fig. 2: Confusion matrices on all unseen test environments, along the four milestones. The contrastive representation (b) softens the $L \to E$ confusion of the baseline (a); the sliding window (c) lifts every class; fusion (d) repairs the low-motion pair itself, L going from 0.49 to 0.71 with the same weights.

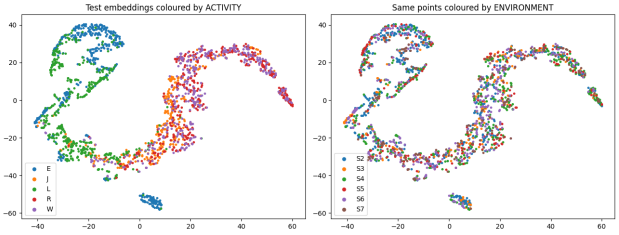


Fig. 3: t-SNE of the SupCon test embeddings (sliding-window run). Left, by activity: a static region (empty, sitting) and a moving arc (jumping, running, walking). Right, the same points by environment: colors are mixed inside the structure, i.e. the space is organized by activity, not by environment.

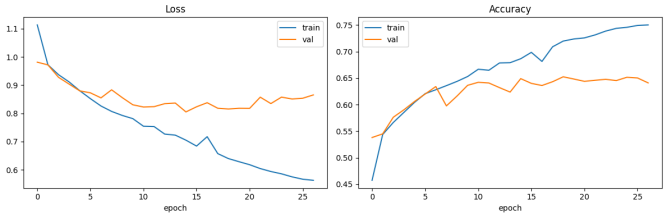


Fig. 4: Train/validation loss and accuracy for the adopted sliding-window schedule (stride 30, patience 12): early stopping restores the epoch-15 weights (lowest validation loss) even though training accuracy keeps climbing through epoch 27.

running, walking), but the activities within each region stay partly entangled: contrastive learning removed the domain nuisance and exposed a harder, intrinsic limit: a single-antenna ≈2 s window does not fully separate the activities, whichever way it is encoded. Consistently, in both runs the SupCon loss fell only from ≈ 4.5 to ≈ 4.3: the pre-training never fully converged.

With sliding-window data, the SupCon probe reaches 59.5%, slightly below the 60.1% baseline, and fine-tuning reaches 58.0%. Thus, contrastive learning helps when training data is limited, but its advantage disappears when many more windows are available.

C. Sliding-window front-end

Cutting the same recordings at stride 30 instead of 340 turns 13,564 windows into 152,320 and takes the unchanged architecture from 48.2% to **60.1**% (+11.9 points, +8.6 to +16.1 per set). Two caveats: the run also raises patience from 7 to 12, so part of the gain belongs to the longer schedule, and the extra windows are correlated and the effective sample size grows far less than tenfold. The cost is ≈21 GB resident and an epoch an order of magnitude slower but the denser windows buy real fitting capacity, not noise: 0.750 training accuracy with validation 12 points above the non-overlapping run.

D. Multi-antenna decision fusion

Averaging the four per-antenna probability vectors (Eq. (2)) is the largest single improvement we measured, and it is free: no retraining, no extra parameters, one softmax and one group-average at inference. On the sliding-window model it lifts 60.1% to **72.7**% (+12.6 points, from +6.0 on $S7$ to +16.9 on $S6$); on the non-overlapping model it gave +10.2 (51.3→ 61.5%) so robust to the data regime.

Its mechanism changes with the regime. At stride 340 fusion sharpened the motion classes but left sitting→empty untouched: with almost no motion the four antennas agreed on the same wrong answer. At stride 30 the single-antenna model is already half-right on that pair (Fig. 2(c), L at 0.49) and fusion turns it into the largest per-class gain: L reaches 0.71, $L \to E$ falls from 0.46 to 0.27 next to jumping

($0.57 \rightarrow 0.73$) and running ($0.65 \rightarrow 0.80$). Averaging helps only where antennas disagree, and denser coverage is what creates that disagreement on low-motion windows.

E. Comparison with the SHARP paper

Tab. 1 places our results next to SHARP [1]: the full system (strongest-path sanitization + 4-antenna fusion) and an ablation that swaps the sanitization for the standard reference-antenna method, feeding the same CNN ([1], Table 4). Full SHARP stays above 95% everywhere. Our best configuration (72.7%) beats the ablation on every unseen set, by 10.4 ($S7$) to 27.2 points ($S5$), with the largest margins exactly where it collapses, that is the blocked-line-of-sight sets. The residual $\sim$26-point gap to full SHARP is thus concentrated in the one ingredient we do not implement.

VI. Reproducing the Experiments

Use the following self-contained notebook to reproduce all reported results and figures:

`NNDL_project_Carraro_Vezzani.ipynb`

It downloads the dataset automatically, so no dataset files need to be uploaded and no paths need to be edited.

1) Upload or open the notebook in Google Colab.
2) Open *Runtime* $>$ *Change runtime type*, then select Python 3, a T4 GPU, High-RAM, and the latest runtime version.
3) Run all cells from top to bottom with *Runtime* $>$ *Run all*.

High-RAM is required with the reported `STRIDE = 30`: the windowed dataset uses approximately 20 GB at steady state and peaks near 31 GB while being built. A standard Colab runtime (approximately 12 GB) will run out of memory in Section 2. If High-RAM is unavailable, set `STRIDE = 340` in that section; this uses non-overlapping windows but produces fewer training samples and the corresponding lower accuracies.

A complete run on a High-RAM T4 takes approximately 45–60 minutes. The notebook fixes the NumPy and PyTorch seeds to 0, both initially and before contrastive training. Small differences can remain because some CUDA kernels are non-deterministic.

VII. Concluding Remarks

Under strict testing on unseen environments, the baseline achieved 48.2% accuracy. Supervised contrastive learning, a slower training schedule, overlapping windows and multi-antenna fusion all improved the performance. The largest gains came from overlapping windows and antenna fusion, which together raised accuracy to $\mathbf{72.7}\%$ without changing the baseline network. Contrastive pre-training also helped when the encoder was kept frozen, but fine-tuning reduced its benefit. Overall, using more of the available temporal and antenna information was the simplest and most effective way to improve cross-environment activity recognition.

References

[1] F. Meneghello, D. Garlisi, N. D. Fabbro, I. Tinnirello, and M. Rossi, "SHARP: Environment and Person Independent Activity Recognition With Commodity IEEE 802.11 Access Points," *IEEE Transactions on Mobile Computing*, vol. 22, pp. 6160–6175, Oct. 2023.

[2] F. Meneghello, N. D. Fabbro, D. Garlisi, I. Tinnirello, and M. Rossi, "A CSI Dataset for Wireless Human Sensing on 80 MHz Wi-Fi Channels," *IEEE Communications Magazine*, vol. 61, pp. 146–152, Sept. 2023.

[3] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised Contrastive Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 18661–18673, Dec. 2020.

[4] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," in *International Conference on Machine Learning (ICML)*, pp. 1597–1607, July 2020.